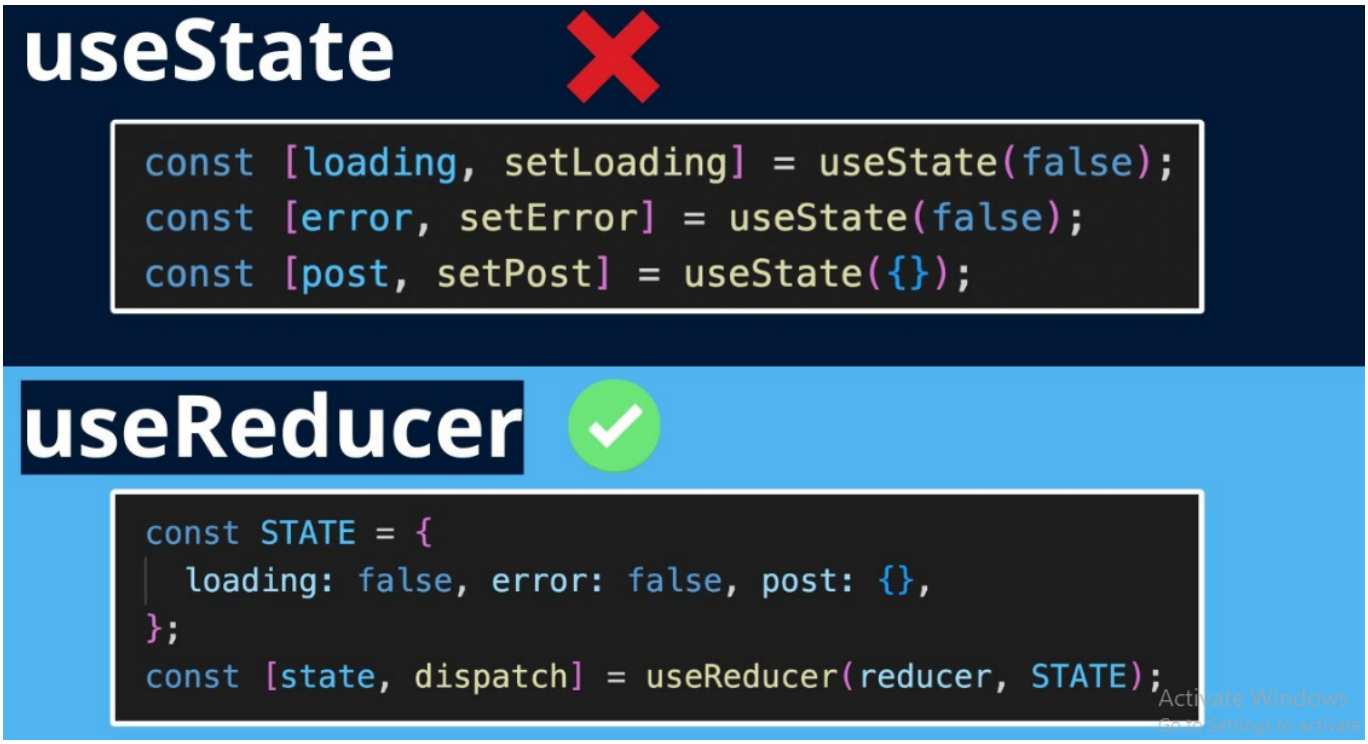
UseReducer

The useReducer hook is an alternative to the useState hook that is preferred when you have complex state logic. It is useful when the state transitions depend on previous state values or when you need to handle actions that can update the state differently.




```
const [state, dispatch] = useReducer(reducer, initialState);
```

- REDUCER : A function that defines how the state should be updated based on the action. It takes two parameters: the current state and the action. Ek function jo decide karta hai ki next state kya hogi.

1. `reducer` Function

- **What it is**: A pure function that determines how the state should change.
- **Parameters**:
  - **Current state**: The existing state value.
  - **Action**: An object that describes what kind of update should happen (usually contains a `type` and optional `payload`).
- **Returns**: The **new state** based on the action.

Example of a `reducer`:

javascript Copy Download

```javascript
function reducer(state, action) {
  switch (action.type) {
    case "INCREMENT":
      return { count: state.count + 1 };
    case "DECREMENT":
      return { count: state.count - 1 };
    case "RESET":
      return { count: 0 };
    default:
      return state; // Return unchanged state if action is unknown
  }
}
```

- INITIAL STATE: The initial value of the state.

2. `initialState`

- The starting value of the state before any actions are dispatched.
- Example: `{ count: 0 }`

- STATE: The current state returned from the useReducer hook.

a) `state`

- The current state value, just like `useState`, but managed by the `reducer`.
- Example: If `initialState = { count: 0 }`, then `state` will initially be `{ count: 0 }`.

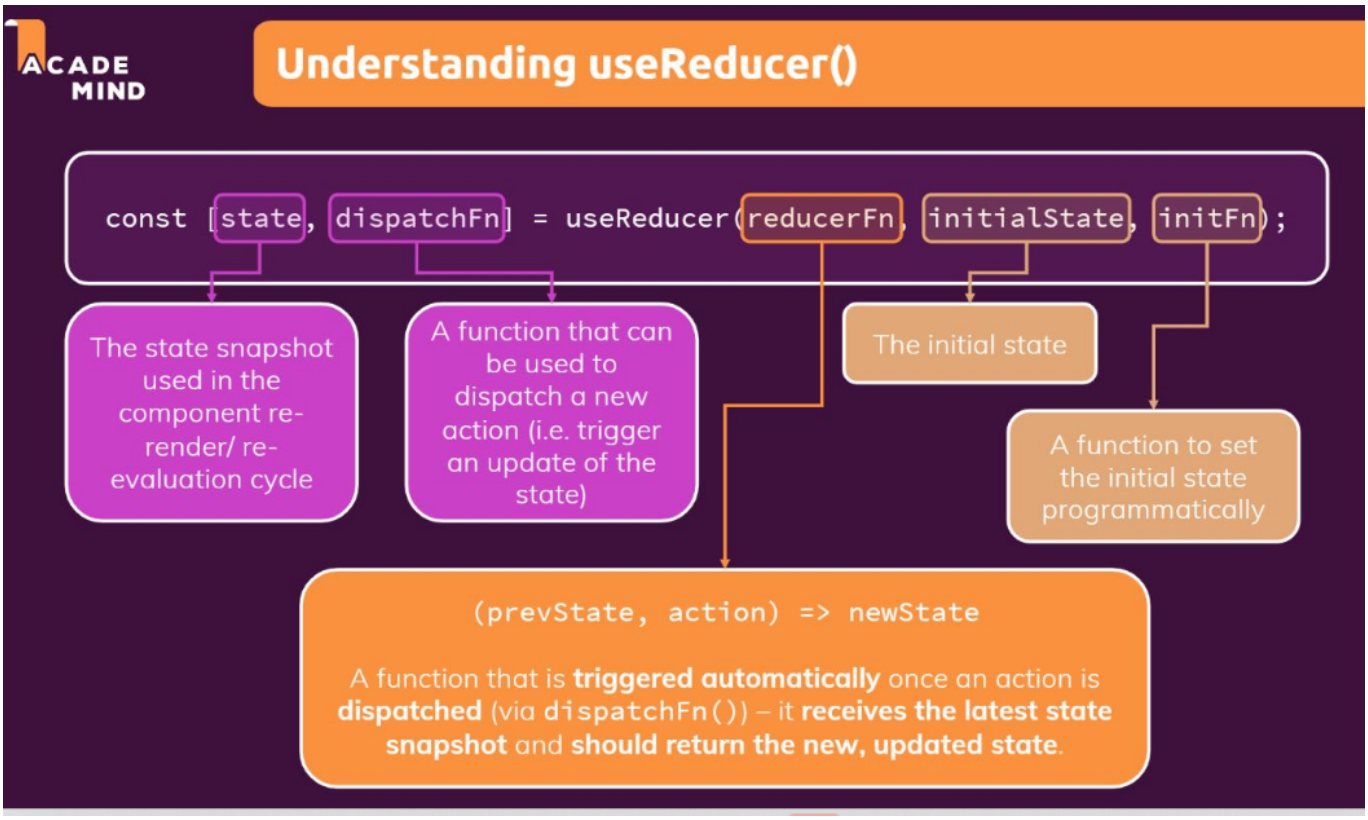
- DISPATCH : A function used to send an action to the reducer to update the state. Function jisse hum action bhejte hain like setState.

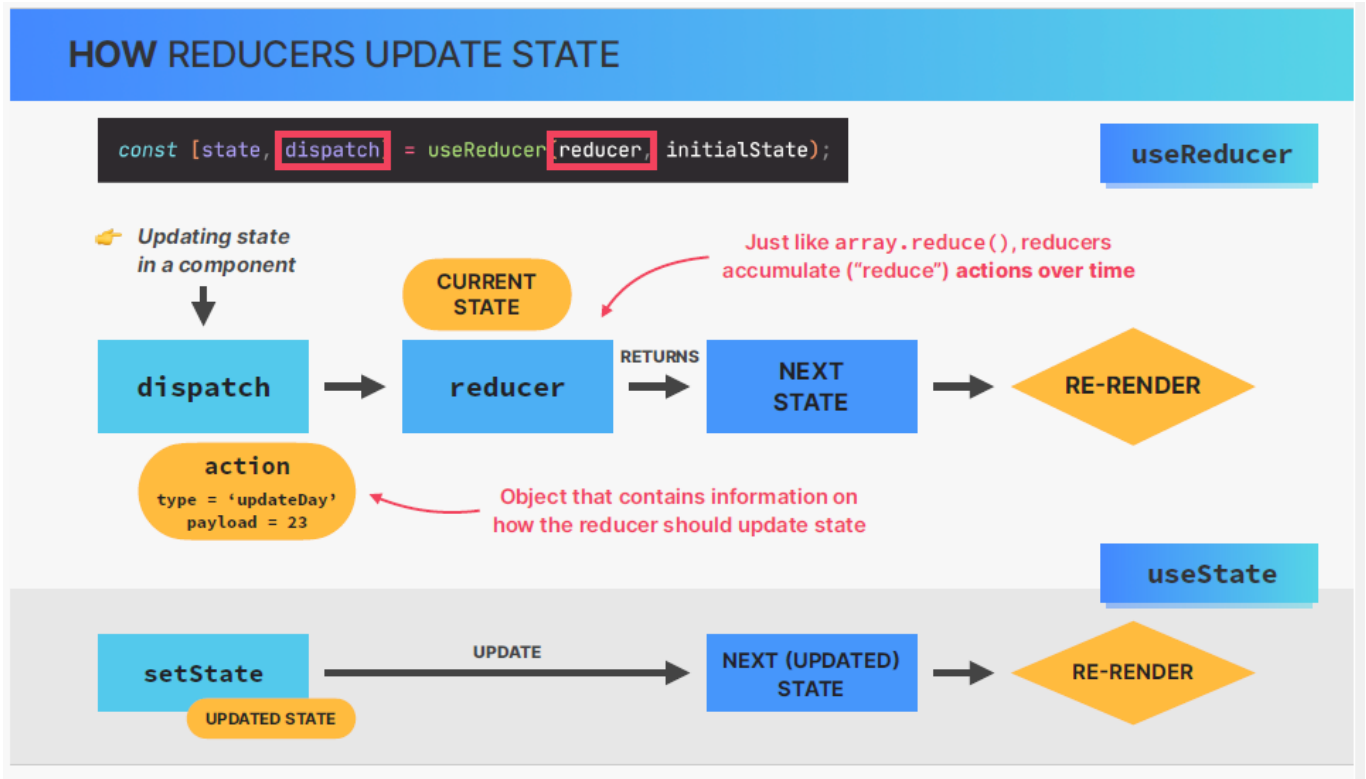
b) `dispatch`

- A function that sends an **action** to the `reducer` to update the state.
- **How to use it**: Call `dispatch({ type: "ACTION_TYPE", payload: data })`.
- Example:

javascript Copy Download

```javascript
dispatch({ type: "INCREMENT" }); // Increases count by 1
dispatch({ type: "DECREMENT" }); // Decreases count by 1
dispatch({ type: "RESET" });     // Resets count to 0
```





EXAMPLE 1: Ek Simple Counter

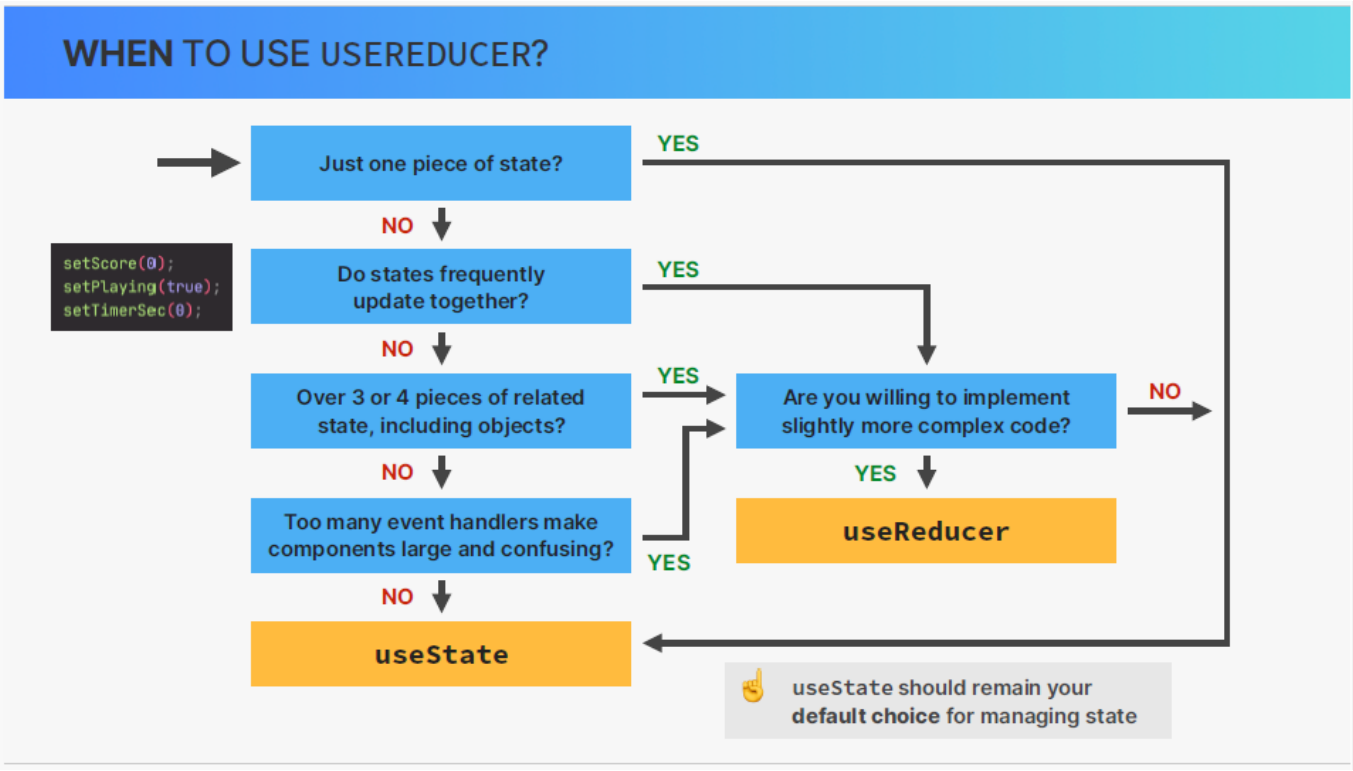
```
import React, { useReducer } from 'react';

const initialState = 0;

function reducer(state, action) {
  console.log('🔄 Reducer called with:', { currentState: state, action });
```

```
  switch (action.type) {
    case 'increment':
      const incrementedState = state + 1;
      console.log('➕ Increment:', state, '→', incrementedState);
      return incrementedState;
    case 'decrement':
      const decrementedState = state - 1;
      console.log('➖ Decrement:', state, '→', decrementedState);
      return decrementedState;
    case 'reset':
      console.log('🔄 Reset:', state, '→', initialState);
      return initialState;
    default:
      console.log('❓ Unknown action type:', action.type);
      return state;
  }
}

function Counter() {
  const [count, dispatch] = useReducer(reducer, initialState);

  console.log('🏠 Component rendered with count:', count);

  const handleIncrement = () => {
    console.log('🖱️ Increment button clicked');
    dispatch({ type: 'increment' });
  };

  const handleDecrement = () => {
    console.log('🖱️ Decrement button clicked');
    dispatch({ type: 'decrement' });
  };

  const handleReset = () => {
    console.log('🖱️ Reset button clicked');
    dispatch({ type: 'reset' });
  };

  return (
    <div className="p-6 max-w-m shadow-md">
      <h1 className="textb-6">Count: {count}</h1>
      <div className="flecenter">
        <button
          onClick={handleIncrement}
          className="px-4 pyors"
        >
          +
        </button>
        <button
          onClick={handleDecrement}
          className="px-4 pors"
        >
          -
        </button>
```

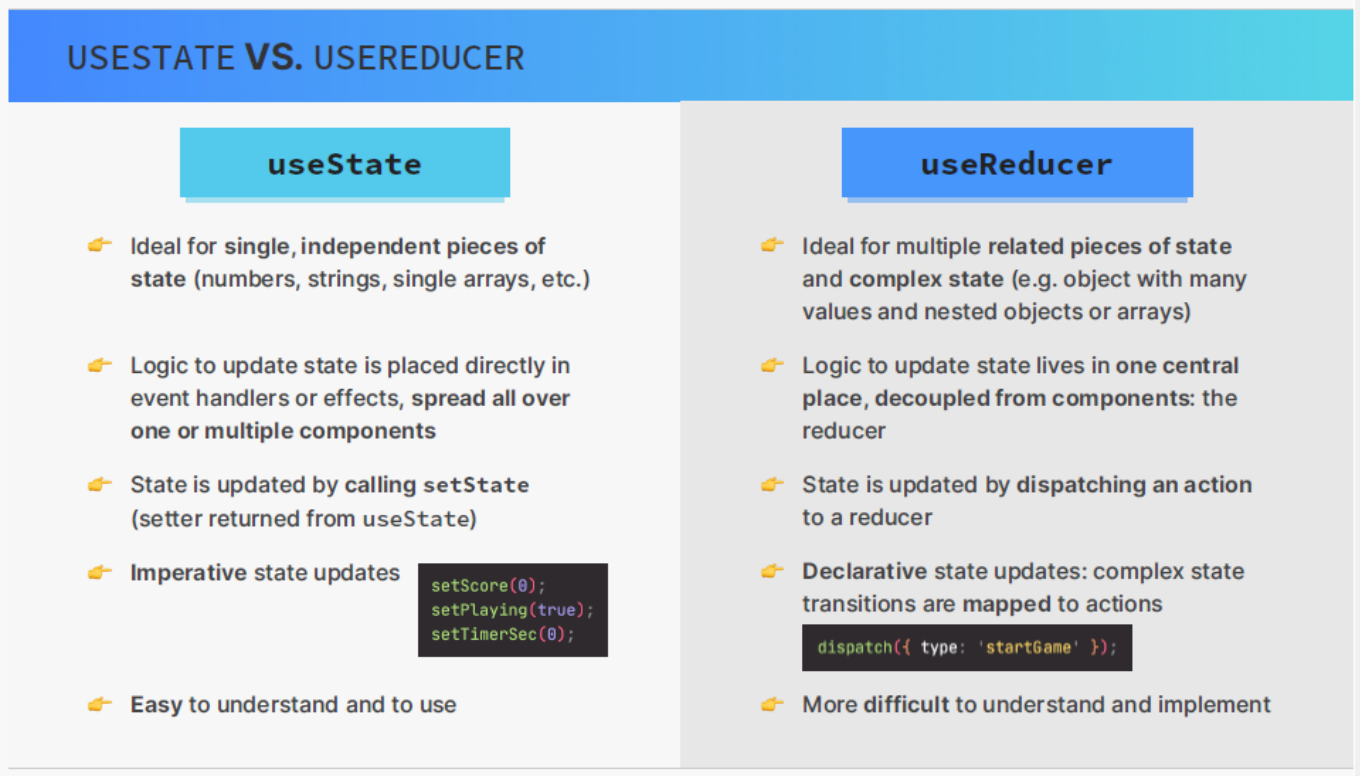
```
          <button
            onClick={handleReset}
            className="px-4 -colors"
          >
            Reset
          </button>
        </div>
        <div className="mt-center">
          Open your browser's developer console (F12) to see the useReducer logs
        </div>
      </div>
    );
}

export default Counter;
```

When to use UseReducer

Scenario	Hook
Simple state (toggle, number)	`useState`
Complex state (object, nested)	`useReducer`
Multiple state updates together	`useReducer`
State logic depends on previous state	`useReducer`
Using Redux-style pattern	`useReducer`



UseState VS UseReducer



EXAMPLE 2 :

```
import { useReducer, useState } from "react";

const initialState = { count: 0, step: 1 };

// RETURN SINGLE VALE
// function reducer(state, action) {
//   switch (action.type) {
//     case "increment":
//       return state + 1;
//     case "decrement":
//       return state - 1;
//     case "setCount":
//       return action.payload;
//     default:
//       return state;
//   }
// }

// RETURN OBJECT HAVING COUNT AND STEP AS WE ARE DEALING WITH MULTIPLE STATE
function reducer(state, action) {
  switch (action.type) {
    case "increment":
      return { ...state, count: state.count + state.step };
    case "decrement":
      return { ...state, count: state.count - state.step };
    case "setCount":
      return { ...state, count: action.payload };
```

```
      case "setStep":
        return { ...state, step: action.payload };
      case "setReset":
        return initialState;
      default:
        return state;
    }
  }

  function DateCounter() {
    const [state, dispatch] = useReducer(reducer, initialState);
    const { count, step } = state;

    // This mutates the date object.
    const date = new Date("june 21 2027");
    date.setDate(date.getDate() + count);

    function dec() {
      return dispatch({ type: "decrement" });
    }

    function inc() {
      return dispatch({ type: "increment" });
    }

    function defineCount(e) {
      return dispatch({ type: "setCount", payload: Number(e.target.value) });
    }

    function defineStep(e) {
      return dispatch({ type: "setStep", payload: Number(e.target.value) });
    }

    function reset() {
      return dispatch({ type: "setReset" });
    }

    return (
      <div className="counter">
        <div>
          <input
            type="range"
            min="0"
            max="10"
            value={step}
            onChange={defineStep}
          />
          <span>{step}</span>
        </div>

        <div>
          <button onClick={dec}>-</button>
          <input value={count} onChange={defineCount} />
          <button onClick={inc}>+</button>
```

```
        </div>

        <p>{date.toDateString()}</p>

        <div>
          <button onClick={reset}>Reset</button>
        </div>
      </div>
    );
  }
  export default DateCounter;
```

The line:

```js
case "increment":
  return { ...state, count: state.count + state.step };
```

is part of your **reducer function**, and here's exactly **what it's doing step by step:**

🔍 What's happening?

This line **creates a new state object** by:

1. **Spreading the existing state** (`...state`) — meaning it copies all current properties (`count`, `step`, etc.) into a new object.
2. **Updating the `count` property** by adding the current `step` value to it.

🔥 Tips & Tricks for `useReducer`

1. ✅ Always return a new state object

Never mutate state directly inside the reducer. Always return a **new object**.

```js
// ✅ Good
return { ...state, count: state.count + 1 };

// ❌ Bad (mutates state)
state.count += 1;
return state;
```

---

2. 🧠 Use action payloads to pass data

Add a `payload` property to actions for flexible updates.

```js
dispatch({ type: 'setStep', payload: 5 });
```

```js
case 'setStep':
  return { ...state, step: action.payload };
```